

2026-06-18 · 过去 24 小时

SGLang-Omni 每日 PR 学习日报

仓库：[sgl-project/sglang-omni](https://github.com/sgl-project/sglang-omni) | 重点：Omni / 多模态 / 多阶段架构

8

合并的 PR 总数

4

深度讲解（核心）

4

简要覆盖（文档/CI）

2

RL 训练对接

CUDA Graph 加速

流式声码器 vocoder

RL rollout 端点

分布式权重热更新 (NCCL)

融合采样 kernel

多阶段 pipeline

Qwen3-Omni 预分词

TTS (MOSS / Higgs)

今日主线：两条 RL PR（#785 rollout + #784 权重同步）把推理引擎接进了强化学习闭环；两条性能 PR（#798 CUDA-graph 声码器 + #816 融合采样）都在攻击 TTS 解码的“启动受限”瓶颈。建议阅读顺序：#798 → #816（性能/CUDA Graph 同源主题）→ #785 → #784（RL 闭环的两半）。讲解深度优先：以下 4 个核心 PR 逐行讲透，文档/CI 类简要覆盖。

[Perf] MOSS-TTS-Local: 用 CUDA Graph 加速

▼ #798 流式声码器 (vocoder) 解码，默认开启且逐字节一致

merged

JiaxinD

多阶段架构 · 性能优化 · 多模态(TTS)

<https://github.com/sgl-project/sglang-omni/pull/798>

1. PR 概览

一句话：把流式语音合成里最慢的一段——声码器逐帧解码——用 CUDA Graph “录制成一条指令回放”，在不改变任何输出字节的前提下，把 c8 并发下端到端延迟降低约 22%。

2. 它解决了什么问题

MOSS-TTS-Local 是一个文本转语音 (TTS) 模型。它的工作分成两大块：

- **AR backbone** (自回归主干)：像普通大语言模型一样，一个 token 一个 token 地“想”出发什么声音 (生成音频码 audio codes)。
- **vocoder / codec** (声码器)：把这些抽象的“音频码”翻译成你真正能听到的波形 (PCM 采样点)。

作者用框架自带的逐请求逐阶段 profiler (`RequestEventRecorder`) 做了测量，发现一个反直觉的结论：在 c8 (同时 8 路请求) 的稳定并发下，声码器这一段占了单请求墙钟时间的 58–63%，反而是最慢、扩展性最差的阶段；而大家以为最贵的 AR 主干只占 24–32%。

为什么声码器这么慢？关键在于流式 (**streaming**) 场景：用户要“边生成边听”，所以服务器每个调度 tick 只解码极少的几帧 (T 很小，最常见 $T=5$)。每解码一小步，都要把一大串 GPU kernel 重新启动一遍。结果是：真正的计算量很小，但“启动 **kernel**”的开销巨大。测量证实了这一点——eager (普通) 模式下每步耗时约 66ms，且在 $T=4$ 到 $T=13$ 之间几乎不变 (如果是算力受限，耗时应该随 T 增长约 6 倍)。这种“时间花在启动而不是计算上”的状态，正是 CUDA Graph 的用武之地。

3. 具体做了什么改动

核心文件：

- `sglang_omni/models/moss_tts_local/streaming_vocoder.py` (+252/-60, 主体逻辑)
- `sglang_omni/models/moss_tts_local/config.py` (+35, 新增配置项)
- `sglang_omni/models/moss_tts_local/vocoder_cuda_graph.py` (新增的 graph runner, 本 PR 引用)

(a) 配置项注入。在 pipeline 配置里新增了三个“model-scoped”开关，并在 `model_post_init` 时把它们注入到声码器工厂参数里——注意作者刻意不用环境变量、不用 **CLI flag**，而是用结构化配置，避免全局污染：

```
cuda_graph: bool = True                # 默认开启
cuda_graph_frames: list[int] | None = None # None = 用内置的 broad-exact
cuda_graph_min_free_gb: float = 3.0      # 显存不足这个量就跳过捕获，安全退

def model_post_init(self, __context=None):
    ...
    for stage in self.stages:
        if stage.factory.endswith("create_vocoder_executor"):
            stage.factory_args.setdefault("cuda_graph", self.cuda_graph)
            stage.factory_args.setdefault("cuda_graph_frames", self.cuda_graph_frames)
            stage.factory_args.setdefault("cuda_graph_min_free_gb", self.cuda_graph_min_free_gb)
```

(b) 让“有状态”的解码也能被 **Graph** 捕获——这是全 **PR** 最精妙的地方。流式解码是有状态的：每个 slot（槽位，代表一路并发请求）都有自己的“因果偏移 / KV 缓存”，跨步骤延续，这样切 chunk 不会在边界处产生杂音。问题是：上游 codec 每一步都把 KV 缓存重新赋值成一个新张量（`state.cached_keys = 新张量`），它的内存地址（`data_ptr`）每步都变。而 CUDA Graph 录制的是固定的内存地址，地址一变，回放就会去读“录制时那一刻”的旧缓存，导致解码结果不一致。修复办法（由 codec 团队 CloudRipple 在 #811 贡献、本 PR 移植）是把缓存改成原地写入（**in-place**）：地址固定、数值与 eager 完全相同，于是 Graph 才能正确捕获。

(c) 录制与回放。在 warmup（预热）阶段，针对每一个会出现的步长 T 各录制一条 graph；服务时只“回放”。下面是 `step()` 的核心——它先尝试 graph 回放，失败则安全退回 eager：

```

graphed = None
graph_failed = False
try:
    with torch.no_grad():
        if self._cg_runner is not None:
            try:
                graphed = self._cg_runner.decode_step(codes_step, exec_m
            except Exception:
                graph_failed = True
                raise
        if graphed is not None:
            audio, audio_lengths = graphed # 走了 CUDA Graph
        else:
            self._codec._set_streaming_exec_mask(exec_mask)
            result = self._codec._decode_frame(codes_step, codes_lengths
            audio, audio_lengths = result.audio, result.audio_lengths
# 一次批量 D2H (device->host) 拷贝。注意：graph 回放的错误可能"异步"地在这里
# 才爆出来（而不是在 decode_step 里），所以输出物化也包在同一个 try 里。
            audio_cpu = audio[slots].detach().to("cpu", torch.float32)
            lengths_cpu = audio_lengths[slots].detach().to("cpu")
except Exception:
    # graph 回放失败：永久禁用 runner，从此走 eager (eager 自己的错误不会禁用它)
    if self._cg_runner is not None and (graph_failed or graphed is not None):
        self._cg_runner = None
    raise

```

(d) 录制哪些 **T**? (用真实数据驱动的工程决策) 这是本 PR 体现“严谨”的部分。作者列举了 5 种候选“捕获集合”，并用真实的 **T** 分布去验证：

方案	graph 数	步骤覆 盖率	是否补零 (padding)	逐字节一致?
focused (4 个关键 T)	3	~45%	无	通过
bucket 向上取整	7	~93%	75% 步骤要补零	失败
2 的幂	6	100%	93% 步骤补零	失败
broad-exact (实测 高频 T + 上限)	13	~87%	无	maxdelta=0
	~23	100%	无	通过

full-exact (每个观测到的 T)				
-----------------------	--	--	--	--

关键洞察：补零会破坏逐字节一致性。如果把一个真实的 $T=5$ 步塞进一个更大的 graph 里再补零，ConvTranspose（转置卷积）的感受野会把补的零拉进边界输出，误差 maxdelta 高达 0.07，后续步还会污染滑动窗口缓存（误差 0.64–0.67）。而恰好那个被补零的 T （占 38% 步骤）是最常见的。所以 $\text{bucket} / 2$ 的幂方案是因为正确性被淘汰，不是因为性能。最终选了 broad-exact：高覆盖、零补零、逐字节一致、显存可行（13 条 graph 仅多占 0.31GB，因为显存主要被那条 $T=100$ 的上限 graph 吃掉）。

4. 涉及的知识（重点）

CUDA Graph 是什么？

它是什么：GPU 上每个运算（矩阵乘、加法、softmax.....）都是一个“kernel”。CPU 要逐个把 kernel “发射”给 GPU，每次发射都有固定的小开销（约几微秒）。CUDA Graph 允许你把“一连串 kernel 发射”预先录制成一张静态的依赖图，之后用一条命令“回放”整张图，省掉成百上千次单独发射的 CPU 开销。

为什么需要它：当单个 kernel 很小、数量很多时（launch-bound，启动受限），CPU 发射开销会超过 GPU 实际算的时间。这正是流式声码器逐帧解码的处境。在本 PR 里：把“每步解码这一长串小 kernel”录成 per- T 的 graph， $T=5$ （最常见）从 65.8ms/步降到 30.7ms/步，2.14 倍加速。

eager 模式 vs graph 模式

eager（即时执行）是 PyTorch 默认行为：写一行算一行，每个算子立刻发射到 GPU。直观、好调试，但每个算子都付发射开销。**graph** 模式是“先录后放”。类比：eager 像每道菜现点现做、每次都跟厨房口头下单；graph 像把整套流程写成一张固定工单，一次性甩给厨房。本 PR 的妙处是即使输出逐字节相同，仅靠省下“下单”开销就快了一倍。

vocoder（声码器）/ codec / RVQ 音频码

它是什么：现代 TTS 不直接生成波形，而是先生成一串离散的“音频 token”（audio codes），再由 codec 解码器还原成波形。这些码通常由 **RVQ**（残差矢量量化，**Residual Vector Quantization**）产生：用多个码本（codebook）逐层逼近，第一层抓粗轮廓，后面几层补残差细节。MOSS 用 RVQ-12（12 个码本，代码里 $n_{vq}=12$ ）。

为什么需要它：离散 token 让“声音”可以像“文字”一样用自回归 LM 来生成。
在本 PR 里：被 CUDA Graph 加速的就是这个 codec 的“把码翻译回波形”这一步。

流式 (streaming / SSE) 与多阶段 pipeline

它是什么：流式指“边生成边返回”，用户不必等整段音频做完。SGLang-Omni 用 **SSE (Server-Sent Events)** 把音频块持续推给客户端。整个服务被切成多个 **stage** (阶段)：preprocess → AR 引擎 → 声码器，各阶段可分配到不同 GPU，像流水线一样并行。

为什么需要它：多阶段流水线能让不同阶段重叠执行、各自扩容，是 Omni (多模态) 服务的核心架构。

在本 PR 里：声码器是其中一个独立 stage；因为流式让它“每步只解码几帧”，才暴露出 launch-bound 瓶颈。

有状态解码 / KV cache / 滑动窗口 与 data_ptr 稳定性

它是什么：自回归/因果模型解码时，会缓存历史的 Key/Value (KV cache)，避免每步重算整段历史；滑动窗口则只保留最近一段。**data_ptr** 是张量底层内存的地址。

为什么这里关键：CUDA Graph 录的是“固定地址上的操作”。如果每步都 new 一个张量来存缓存，地址变了，graph 回放就读错了内存。

在本 PR 里：把缓存改成“原地写入 (地址不变、数值不变)”，是让“有状态流式解码”能被 graph 捕获的前提 (#811)。

Little's 定律：延迟与吞吐的关系

它是什么：排队论里 $L = \lambda \times W$ —— 系统中平均请求数 = 到达率 × 平均逗留时间。

在本 PR 里：作者诚实地指出：声码器段延迟下降 (c8 下 -40%) 是“可信、可复现”的硬信号；而吞吐提升 (+16%~+29%) 只是这个延迟下降经 Little 定律换算出来的结果，依赖机器能否撑满并发，所以给的是一个区间而非单一数字。这是一种很专业的“区分可信信号与派生信号”的表达。

5. 我能学到什么 / 延伸思考

- 先测量再优化。作者没有凭直觉去优化 AR 主干，而是用 profiler 定位到“反直觉”的真瓶颈 (声码器)，这是性能工程的第一原则。

- 正确性是优化的硬约束。整个 PR 反复强调“bit-for-bit identical (逐字节一致)”，并用它把几个高覆盖率方案直接淘汰。优化不能以牺牲输出为代价——这点尤其适用于音频/模型这类“肉眼难发现退化”的领域。
- **fail-safe** 设计。每条失败路径（显存不足、捕获 OOM、回放出错、形状没录过）都安全退回 eager，“没有加速但也绝不出错”。这是把激进优化默认开启的底气。
- 延伸前置知识：CUDA 编程模型（kernel/stream）、PyTorch 的 `torch.cuda.CUDAGraph` 与 `capture_begin/replay`、转置卷积的感受野、TTS 的 codec/RVQ 原理。

▼ #785 [RL] 为 Qwen3-Omni 增加 Miles 兼容的 / generate rollout 端点

merged

yxs Omni(多模态) · RL 强化学习 · 推理-训练对接

<https://github.com/sgl-project/sglang-omni/pull/785>

1. PR 概览

一句话：给推理服务器加一个 POST /generate 端点，让强化学习训练器 (Miles) 能用一次调用驱动生成，并拿回“采样时每个 token 的 logprob”，从而把推理引擎接进 RL 训练闭环。

2. 它解决了什么问题

这是把 SGLang-Omni 接入 **RL** (强化学习) 训练闭环的“推理侧”工作。在 RLHF 类训练里有两个角色：

- **trainer** (训练器，这里是 **Miles**)：负责更新模型权重 (policy/策略)。
- **inference engine** (推理引擎，就是 **SGLang-Omni**)：负责用当前权重“采样生成”大量样本 (这一步叫 **rollout**)，给训练器算梯度用。

痛点有两个：

1. 没有合适的端点。之前 Qwen3-Omni 的预处理是“只认 messages (聊天消息)”的；如果 Miles 直接发它训练用的 `input_ids` (已经分好词的 token)，服务器会在 `normalize_messages` 处直接 500 报错。
2. 拿不到“正确的”**logprob**。RL 更新策略时需要知道“当时这个 token 是以多大概率被采样出来的” (logprob)。如果事后用原始 logits 重算，会丢失温度 (temperature) 等采样语义，算出来的 logprob 就不属于“真正采样它的那个策略”，训练就会有偏差。

3. 具体做了什么改动

这个 PR 把 rollout 所需的数据从 HTTP 层一路打通到 model runner，再原路返回。核心文件：`serve/openai_api.py` (+189, 新端点)、`serve/protocol.py` (请求/响应 schema)、`models/qwen3_omni/components/preprocessor.py`

(预分词分支)、`model_runner/base.py` (记录采样 `logprob`)、`client/client.py` 与 `router` (透传)。

(a) 预分词 (pre-tokenized) 分支：让 `input_ids` 直达 `thinker`。新增一个判断：如果输入是“一串纯整数”，就认定为预分词请求，跳过聊天模板和 HF `processor`，直接用这串 token 构建推理状态。这样 Miles 发什么 token、模型就吃什么 token，不会因为服务器端重新分词而产生“训练/推理 token 漂移”。

```
def _is_pretokenized_prompt(inputs):
    """True when a rollout request carries pre-tokenized prompt ids."""
    return (
        isinstance(inputs, list)
        and bool(inputs)
        and all(isinstance(token, int) for token in inputs)
    )

# __call__ 入口处：
if _is_pretokenized_prompt(inputs):
    return self._preprocess_pretokenized(payload, inputs)
# 否则走原来的 messages / dict 路径
```

预分词分支里直接用这串 token 造张量、造 `attention_mask`，并把图像/音频编码器标记为“跳过”（因为这条路只处理纯文本；多模态 `input_ids` 仍走 `messages` 路径）：

```
def _preprocess_pretokenized(self, payload, token_ids):
    input_ids = torch.tensor(token_ids, dtype=torch.long)
    attention_mask = torch.ones_like(input_ids)
    validate_prompt_seq_len(input_ids, max_seq_len=self.max_seq_len, ...)
    return self._finalize_state(
        payload,
        input_ids=input_ids,
        attention_mask=attention_mask,
        prompt_text="",
        full_mm_inputs={},
        encoder_inputs={
            # 显式跳过两个编码器
            "image_encoder": {"_skip": True, "_result": {}},
            "audio_encoder": {"_skip": True, "_result": {}},
        },
    )
```

顺手做了一个不错的重构：把“组装 thinker 状态”的逻辑抽成

`_finalize_state()`，让预分词分支和原 messages 分支共用同一处“形状的唯一来源”，避免两条路各写一遍、日后不一致。

(b) logprob 来自采样器，而不是事后重算。在 `_sample_next_token_ids` 里，如果有请求要 logprob，就先打开采样器的 logprob 开关，采样后直接读采样器产生的 `next_token_logprobs`，逐请求追加进 `output_token_logprobs`。这保留了“真正采样它的那个策略”的概率语义：

```
wants_rollout_logprob = any(sr.data.return_logprob for sr in requests)
if wants_rollout_logprob:
    self._enable_sampler_logprobs(forward_batch, len(requests)) # 打开
next_token_ids = self.tp_worker.model_runner.sample(logits_output, forward_batch)
if wants_rollout_logprob:
    next_token_logprobs = logits_output.next_token_logprobs # 采样
    self._record_rollout_logprobs(next_token_logprobs, next_token_ids, requests)
```

而 `_record_rollout_logprobs` 里有一个非常重要的批次对齐校验——并发批处理时，第 *i* 行的 logprob 必须配第 *i* 行的 token，否则就是“串台”（batch cross-contamination）。代码显式检查三者长度一致，再逐行配对：

```
logprobs = sampled_logprobs_to_list(next_token_logprobs)
token_ids = [int(t) for t in next_token_ids.tolist()]
if len(logprobs) != len(token_ids) or len(logprobs) != len(requests):
    raise RuntimeError("rollout logprob batch-size mismatch: ...")
for row_idx, sched_req in enumerate(requests):
    data = sched_req.data
    if data.return_logprob:
        data.output_token_logprobs.append([logprobs[row_idx], token_ids[row_idx]])
```

(c) 一些“看似小但很重要”的契约设计：

- 计数字段（`prompt_tokens` / `completion_tokens` / `cached_tokens`）永远序列化成 **int**（未知时为 0），绝不 **null**——因为 RL 消费方会做 `cached_tokens += meta_info.get("cached_tokens", 0)`，一旦是 `None` 就会 `int + None` 崩溃。
- 空的 `output_token_logprobs = []` 被保留，与“没采集到 logprob（null）”是两种不同语义——零生成 token 的 rollout 是合法值。
- `omni_rollout` 字段被“前置声明”但本 PR 不填充——多模态可训练动作容器是下一步，这里先把管道铺好。

4. 涉及的知识点（重点）

RL rollout（强化学习采样）与 RLHF 闭环

它是什么：在用 RL 微调大模型时，需要让当前策略模型大量“试着生成”（rollout），再根据奖励信号更新权重。rollout 由推理引擎做，权重更新由训练器做，两者反复循环。

为什么需要它：推理引擎采样又快又省，但要把“它采了什么、概率多少”精确回传给训练器，闭环才成立。

在本 **PR** 里：/generate 端点就是这个闭环的“推理侧入口”，回传 output_token_logprobs 供策略更新。

logprob（对数概率）与为什么不能事后重算

它是什么：模型对每个候选 token 给一个概率，取对数就是 logprob (≤ 0)。

为什么重要：RL 的策略梯度需要“采样时刻该 token 的概率”。采样会经过 temperature/top-p 等变换，事后用原始 logits 重算得到的是“另一个分布”的概率，不等于真正采样它的那个策略——会引入偏差。

在本 **PR** 里：直接记录采样器输出的 next_token_logprobs，保证 logprob 与“实际采样动作”严格对应。

tokenizer / input_ids / chat template / 训练-推理 token 漂移

它是什么：tokenizer 把文字切成整数 token (input_ids)；chat template 把聊天消息拼成模型期望的格式再分词。

为什么这里关键：如果训练器用 token 序列 A 算梯度，而推理服务器把同样的文字重新分词成了略不同的 B，二者“看到的输入”就不一致（token drift），RL 会学歪。

在本 **PR** 里：预分词分支让服务器直接吃训练器发来的 input_ids，绕开模板和分词器，从源头消除漂移。验证中也证明：同一 prompt 用 input_ids 和用 messages 发，贪心解码产生完全相同的 token-id。

Qwen3-Omni 的 thinker / 多模态编码器

它是什么：Qwen3-Omni 是多模态模型，文本/图像/音频先各自经 encoder 编码，再交给“thinker”（负责推理生成的主干）。

为什么需要 **encoder**：图像/音频要先变成 thinker 能消化的 embedding。

在本 **PR** 里：纯文本 rollout 不需要图像/音频，所以预分词分支显式把两个 encoder 标 `_skip`，让 `input_ids` 直达 `thinker`。

continuous batching / 批内行对齐

它是什么：推理服务把多路请求拼成一个 batch 同时算（continuous batching 是 SGLang 等引擎的核心吞吐手段）。batch 里第 i 行对应第 i 个请求。

为什么这里关键：若 `logprob/token` 的行序和请求序错位，就会“张冠李戴”。

在本 **PR** 里：显式断言 `logprobs`、`token_ids`、`requests` 三者长度一致并逐行配对，并用 8 路并发测试验证每路的 `logprob` 解码回的正是自己的回复文本。

5. 我能学到什么 / 延伸思考

- “契约思维”。一个对外端点的价值不只在功能，更在它的字段语义是否严谨：`int` 永不为 `null`、空列表 $\neq$ `null`、长度等校验……这些细节决定了下游能不能放心 `+=`。
- 从数据源头消除偏差。与其在下游修补 `token` 漂移，不如让服务器直接接受训练器的原始 `token`——把问题消灭在源头。
- 端到端打通 **vs** 一次到位。omni_rollout 先“铺管道、不填充”，是大型系统里常见的渐进式演进策略。
- 延伸前置知识：策略梯度 / PPO / GRPO 的基本原理，HF tokenizer/chat template，SGLang 的 `sampler` 与 `forward_batch` 结构。

▼ #784 [RL] 分布式权重同步 (distributed weight-sync)

merged

yxs 多阶段架构 · RL 强化学习 · 分布式(NCCL)

<https://github.com/sgl-project/sglang-omni/pull/784>

1. PR 概览

一句话：让外部训练器能通过 NCCL 把更新后的权重“热推”进正在运行的 Omni 服务器——不重启、不落盘，是 RL 闭环的另一半（推理侧权重刷新）。

2. 它解决了什么问题

承接 #785 (rollout 端点)，这个 PR 解决 RL 闭环的另一半：训练器更新完权重后，怎么把新权重送进正在服务的推理引擎？

之前的三个分布式端点都是“占位”状态，调用返回 `501 Not Implemented`。最朴素的做法是“存 checkpoint 到磁盘 → 推理引擎重启加载”，但这在 RL 里太慢了——RL 每隔几步就要刷新一次权重。本 PR 实现的是 **live refit**（在线权重热更新）：训练器在 rank-0 上，把刚更新的策略权重通过 **NCCL** 广播给每个推理副本，推理引擎不重启就吞下新权重。

3. 具体做了什么改动

(a) 关键设计：控制面只传元数据，张量走专用 **NCCL** 通道。HTTP/IPC 控制路径只携带“参数名 / dtype / shape / group 名”这些元数据；真正的权重张量通过一个专用的 NCCL 进程组传输，绝不挤进 HTTP。三个端点被一路打通：

`openai_api(HTTP) → Client → Coordinator.admin → Stage runtime → OmniScheduler → ModelWorker → sglang ModelRunner`

端点	作用
<code>POST /init_weights_update_group</code>	和训练器握手，建立 NCCL 组。推理 $\text{rank} = \text{rank_offset} + \text{tp_rank}$ 。

<code>POST / update_weights_from_distributed</code>	接收广播来的张量，按 (names, dtypes, shapes, group_name) 对应。
<code>POST / destroy_weights_update_group</code>	拆除该组。

(b) ModelWorker 侧的“防御性长度校验”。这是个很有教益的细节：sglang 底层用 `zip(names, dtypes, shapes)` 来配对，而 Python 的 `zip` 遇到长度不等会静默截断到最短——那样会“少广播一些权重”而不报错，是个隐蔽的 bug。所以 worker 在调底层前显式校验三者非空且等长：

```
names = payload.get("names"); dtypes = payload.get("dtypes"); shapes = p
if names is None or dtypes is None or shapes is None:
    return False, "names, dtypes and shapes are required"
# Pydantic 已在 HTTP 边界保证了类型/非 None；这一条长度校验才是最关键的一
# sglang 会 zip 三者并静默截断到最短，导致权重“少广播”。
if len(names) != len(dtypes) or len(names) != len(shapes):
    return False, "names, dtypes and shapes must have the same length"
success, message = update(names, dtypes, shapes,
                           payload.get("group_name") or "weight_update_group",
                           load_format=payload.get("load_format"))
```

(c) 调度器侧：**refit** 与生成“串行化”，并抽象出生命周期辅助函数。admin 请求在调度循环线程的每个 tick 开头被排空处理（`_process_admin_requests()`），这样一次 refit 是和生成串行的，不会和正在跑的请求竞态。默认情况下，有请求在飞行（**in-flight**）时拒绝 **refit**。本 PR 还把原本写死在 disk-update 里的“暂停引擎 → 排空 → 更新 → flush 缓存 → 恢复”这套生命周期，抽成可复用的 `_run_weight_update_with_lifecycle()`，让 disk / distributed 两条路共用：

```
def _run_weight_update_with_lifecycle(self, payload, update_fn, result_callback,
                                     keep_pause_on_failure=False):
    keep_pause = bool(payload.get("keep_pause", False))
    with self._admin_lock:
        previous_pause_state = self._engine_paused
        self._engine_paused = True          # 先暂停引擎
    ...
    # 若有活跃请求且不允许带请求更新 -> 拒绝，并据 keep_pause 决定是否恢复原状态
    ...
```


(d) **Router** 单副本守卫。 `/init_weights_update_group` 在目标多于一个存活 worker 时直接拒绝 (422)。原因：每个副本需要不同的 **rank_offset**，把一次 init 扇出给 N 个副本会造成 rank 冲突。多副本 refit 明确不在本 PR 范围内。

(e) 验证亮点：**bit-parity** (逐张量 **SHA256** 比对)。测试加载一个 instruct 模型，通过分布式路径把它 refit 成 base checkpoint，然后断言：每一个被 **refit** 改动的推理参数，都和“直接从 **base** 加载的服务器”逐张量 **SHA256** 一致。注意比对是“推理 vs 推理”，而不是“推理 vs 原始 checkpoint”——因为模型在 `load_weights` 时会重命名和融合参数 (397 个 `body.*` checkpoint 张量会塌缩成约 226 个融合参数)，checkpoint 和服务模型根本不共享 key。实测 2×H100 上 refit 全部 397 个 `body.*` 参数耗时 0.868s，全部 SHA256 对齐，刷完还能继续出音频。

4. 涉及的知识点 (重点)

NCCL 与集合通信 (**collective communication**)

它是什么：NCCL (NVIDIA Collective Communications Library) 是多 GPU 间高速通信库，提供 broadcast (广播)、all-reduce 等“集合操作”，走 NVLink/InfiniBand，比走 CPU/HTTP 快几个数量级。

为什么需要它：权重动辄几 GB，必须走 GPU 间直连通道，不能塞进 HTTP。

在本 PR 里：训练器 rank-0 用 NCCL broadcast 把权重发给每个推理 rank；控制面 (HTTP) 只传元数据来协调这次广播。

控制面 vs 数据面 (**control plane / data plane**) 分离

它是什么：系统设计常把“发命令/协调”的控制面与“搬大数据”的数据面分开。

为什么需要它：控制面要轻、要可靠；数据面要快。混在一起会互相拖累。

在本 PR 里：HTTP 控制面只带 names/dtypes/shapes/group_name 元数据；GB 级张量走独立 NCCL 数据面。这是非常典型的工业级架构选择。

tensor parallelism (张量并行) / rank / rank_offset

它是什么：大模型放不进单卡，就把每层的权重矩阵“切片”分到多张卡上协同算，这叫张量并行 (TP)。每张卡有一个 rank (编号)。

为什么需要 **rank_offset**：训练器和推理引擎的 rank 要拼进同一个全局 NCCL 组，推理 $\text{rank} = \text{rank_offset} + \text{tp_rank}$ 才不冲突。

在本 **PR** 里：多副本会各自需要不同 `rank_offset`，所以 `router` 用单副本守卫防止 `rank` 撞车。

多阶段 Omni 架构的 admin 控制面

它是什么：SGLang-Omni 不是单进程，而是 `coordinator`（协调器）→ `stage`（阶段）→ `scheduler`（调度器）→ `model worker` 的多层结构。

为什么这里关键：一个 `refit` 命令必须穿透每一层才能到达真正持有权重的 `ModelRunner`。

在本 **PR** 里：把三个端点沿 `coordinator.admin` 这条 `admin` 控制链一路 `plumb`（铺管）到底，并在调度器线程内串行执行，避免与生成竞态。

参数融合/重命名 与 SHA256 bit-parity 验证

它是什么：加载权重时，框架常把多个小权重融合成一个大算子的参数（如 QKV 三个矩阵融合成一个），并重命名 `key`。

为什么影响验证：这样 `checkpoint` 文件里的 `key` 和内存里服务模型的 `key` 不再一一对应，不能直接比文件。

在本 **PR** 里：改用“推理 vs 推理”逐张量 SHA256 比对——证明的是“权重确实变成了 `base` 的样子”，而不仅仅是“校验和变了”。这是一种严谨的等价性验证思路。

5. 我能学到什么 / 延伸思考

- 识别第三方库的“静默陷阱”。`zip` 静默截断是个经典坑；作者用一行注释 + 一个显式校验把它挡在门外。读别人代码时，要警惕这种“不报错但行为错”的接口。
- 把横切的生命周期抽成可复用函数。`disk-update` 与 `distributed-update` 共享“暂停→排空→更新→flush→恢复”，抽出 `_run_weight_update_with_lifecycle` 后两条路都受益、也更难写错。
- 串行化是并发正确性的朴素武器。把 `refit` 放到调度循环里和生成串行，避免了一大类竞态——简单但有效。
- 验证要证明“对的事”，而非“变了”。SHA256 推理-vs-推理 `bit-parity` 比“校验和变了”强得多。
- 延伸前置知识：NCCL/torch.distributed 进程组、TP 切分与权重命名、PPO/GRPO 训练循环里推理引擎的角色。

▼ #816 [Higgs] 用 `sgl_kernel` 的 `renorm` 融合 `top-k/top-p` 采样

merged

BBuf 多模态(TTS) · 性能优化 · GPU kernel

<https://github.com/sgl-project/sglang-omni/pull/816>

1. PR 概览

一句话：把 Higgs Audio v3 TTS 解码里“整词表排序 + `topk` + `cumsum` 掩码”的采样实现，换成 `flashinfer/sgl_kernel` 的融合 `renorm` kernel，数值等价但更省 GPU 时间。

2. 它解决了什么问题

Higgs Audio v3 也是 TTS 模型，每个解码步要对多个码本（8 个 **codebook**，词表 **V=1026**）做 `top-k/top-p` 采样。profiling 发现：原实现每步都做一次对整个词表的 `torch.sort` + `topk(K_MAX)` + `cumsum/scatter` 掩码。对这么小的词表（1026）来说，这是一长串“启动受限（launch-bound）”的小 kernel（profiler 里能看到 `radixSortKVInPlace`、`gatherTopK`、`softmax_warp_forward` 等），白白付出排序的开销。

3. 具体做了什么改动

只改一个文件 `sglang_omni/models/higgs_tts/sampler.py`（+20/-17）。核心是在 `_sample_independent_batched` 里，用 `sgl_kernel.top_k_renorm_prob` / `top_p_renorm_prob` 这两个融合 **kernel** 替换掉基于排序的掩码逻辑，贪心/argmax 短路和 graph-safe 的 multinomial 保持不变。改动前后对比：

```
# 改动前：先 softmax，再对整词表排序做 top-p 掩码
if top_p is not None:
    sorted_logits, sorted_indices = torch.sort(logits, descending=True,
        cum_probs = sorted_logits.softmax(dim=-1).cumsum(dim=-1)
    remove = cum_probs > thresh
    remove[..., 1:] = remove[..., :-1].clone()
    remove[..., 0] = False
    scatter = torch.zeros_like(remove)
```

```
scatter.scatter_(-1, sorted_indices, remove) # 一长串排序/散射操作
logits = torch.where(scatter, float("-inf"), logits)
probs = logits.softmax(dim=-1)
codes_flat = probs.reshape(B * N, V).multinomial(num_samples=1).squeeze(-1)
```

```
# 改动后：先算 probs (强制 contiguous fp32)，用融合 kernel 重归一化
probs = logits.float().softmax(dim=-1).reshape(B * N, V).contiguous()
if top_k_buf is not None:
    tk = top_k_buf.view(B, 1).expand(B, N).reshape(B * N).clamp(min=1, max=V)
    tk = tk.to(torch.int32).contiguous()
    probs = _fused_top_k_renorm(probs, tk) # 一个融合 kernel
if top_p is not None:
    tp = top_p.view(B, 1).expand(B, N).reshape(B * N).to(torch.float32)
    probs = _fused_top_p_renorm(probs, tp) # 一个融合 kernel
codes_flat = probs.multinomial(num_samples=1).squeeze(-1)
```

两个值得记住的细节：

- 必须 **contiguous fp32**。flashinfer 的 renorm kernel 要求输入是连续内存的 fp32，否则会静默给出错误结果（不报错）。所以代码处处 `.to(torch.float32).contiguous()`。
- 数值等价性是被证明的。在 $B \in \{1, 4, 16\} \times \text{temperature} \in \{0.7, 1.0, 1.3\} \times \text{top_k} \times \text{top_p}$ 共 225 组配置上对比旧路径：最大概率差仅 $4.8e-7$ （fp32 机器精度），保留的 token 集合完全相同。唯一理论差异在“cumsum 恰好等于 top_p”的边界（measure-zero，连续 logits 下概率为 0，扫描中从未触发）。

性能：H100 上 Higgs 解码每步 GPU 时间从 $4152.1\mu\text{s}$ 降到 $4082.8\mu\text{s}$ （-1.67%），采样 kernel 块（radixSort+gatherTopK+softmax）从约 $64\mu\text{s}/\text{步}$ 降到约 $32\mu\text{s}/\text{步}$ 。CUDA graph 捕获和音频输出不受影响。

4. 涉及的知识点（重点）

top-k / top-p (nucleus) 采样

它是什么：生成时不总取概率最高的 token，而是从一个截断分布里随机采。top-k 只保留概率最高的 k 个；top-p (nucleus) 保留累计概率达到 p 的最小集合。

为什么需要它：控制随机性/多样性，避免重复或胡言。

在本 PR 里：对 8 个音频码本各做 top-k/top-p，决定下一帧音频 token。

kernel 融合 (fused kernel) 与 launch-bound

它是什么：把多个小 GPU 算子合并成一个 kernel，减少启动次数和中间张量读写。launch-bound 指瓶颈在“启动 kernel”而非“算”。

为什么需要它：小词表上“排序+topk+cumsum+scatter”是好几个小 kernel，启动开销占大头。

在本 PR 里：top_k_renorm_prob / top_p_renorm_prob 把“截断 + 重归一化”融成单个 kernel，采样块时间砍半。

sgl_kernel / flashinfer

它是什么：sgl_kernel 是 SGLang 的高性能算子库（很多来自 flashinfer），提供注意力、采样、归一化等手写优化 kernel。

为什么需要它：PyTorch 通用算子在特定场景（如小词表采样）不够快；专用 kernel 更优。

在本 PR 里：直接调用其 renorm_prob kernel 替换 PyTorch 的 sort 路径。

renorm（重归一化）与“概率域”操作

它是什么：top-k/top-p 截断后，留下的概率之和不再是 1，需要重新归一化。融合 kernel 直接在“概率域”（softmax 之后）做截断+归一化。

对比旧法：旧法在“logits 域”用 -inf 掩码再 softmax；新法直接对 probs 动手，省去排序与散射。

在本 PR 里：这就是为什么先 softmax 成 probs、再喂给 renorm kernel。

数值等价性验证 / measure-zero 边界 / 内存连续性陷阱

它是什么：替换实现时要证明“结果不变”。本 PR 做了 225 组 sweep，最大误差 $5e-7$ 。唯一差异在 $\text{cumsum} = \text{top_p}$ 的精确边界，这在连续概率下“测度为零”（几乎不可能发生）。

连续性陷阱：很多高性能 kernel 假设输入内存连续且为特定 dtype，非连续输入会“静默出错”——这是用底层 kernel 时必须警惕的。

在本 PR 里：处处 `.contiguous()` + fp32，并把“边界差异”诚实写进 PR。

5. 我能学到什么 / 延伸思考

- “等价替换”要有证据。不是“我觉得一样”，而是 225 组 sweep + 误差量级 + 唯一差异点的理论分析。这是替换核心路径时该有的严谨度。
- 了解你的硬件抽象的“隐含契约”。contiguous/fp32 这种隐性要求一旦违反会静默出错，比崩溃更难查。

- 小优化也值得做，但要诚实标注收益。 -1.67% 端到端、采样块减半——作者如实报告，不夸大。
- 延伸前置知识：flashinfer 采样 kernel 原理、PyTorch 内存布局 (stride/contiguous)、CUDA graph 对“图内算子”的要求。

其余 PR 简要覆盖 (文档 / CI)

▼ #792 docs: 新增 MOSS-TTS-Local cookbook

merged

MelodyyyYin 文档

做了什么：为 MOSS-TTS-Local 新增用户向使用手册：架构介绍（帧内解码、无 delay pattern、RVQ-12、48kHz 立体声、并发16下 RTF<1）、服务配置（checkpoint 自动解析 MossTTSLocalModel，serve 只需 --model-path）、语音合成（基础/克隆/流式/时长控制/风格标记）、参数表与已知限制。纯文档，1 个新文件 + 1 行 toctree。

能学到：虽然是文档，但它系统梳理了 MOSS-TTS-Local 的架构概念（RVQ-12 码本、帧内 vs delay-pattern 解码、codec/vocoder 分卡部署），是理解 #798 的好背景材料。值得注意的工程文化：作者把无法在仓库中证实的“12.5 Hz 帧率”一项主动删掉——文档也要可追溯、不臆造。

▼ #819 [Docs] 修正 MOSS-TTS-Local 流式命令格式

merged

SandyLuXY 文档

做了什么：修正 cookbook 流式示例：管道给 ffmpeg 的命令必须带 "stream_format": "audio"，否则裸 PCM 流可能产出噪声样的 WAV 甚至触发 ffmpeg PCM 解码错误（即使命令退出码为 0）。附带实测对比（mean_abs 0.59 噪声 vs 0.06 语音）。

能学到：一个“命令退出码 0 却结果错误”的经典坑——提醒我们：退出码成功 ≠ 结果正确。流式裸 PCM 必须显式声明 stream_format，否则字节流被错误解释成噪声。

▼ #817 [CI] 支持定向的 TTS CI 重跑标签

merged

Ratish1 CI/工程

做了什么：扩展 /tag-run-ci-label 与 /tag-and-rerun-ci 斜杠命令，接受 higgs / moss 可选参数，映射到 run-higgs / run-moss 标签（同时仍加 run-ci），并移除对立的 TTS 模型标签以免 workflow 门控同时看到两者。

能学到：了解 SGLang 的 CI 是“标签驱动”的：维护者打 run-ci 标签才在自托管 GPU runner 上跑测试，run-higgs/run-moss 用来定向选择 TTS 预设。这是大型 GPU 项目控制昂贵 CI 成本的常见做法。

▼ #814 [CI] 为 TTS CI 启用标签

merged

zhaochenyang20 CI/工程

做了什么：为 TTS CI 启用 run-higgs / run-moss 标签机制（#817 的前置）。

能学到：与 #817 同一主题：把 CI 资源按模型维度切分、按需触发。

本日报由自动化学习例程生成 · 数据源 : GitHub sgl-project/sglang-omni · 生成于 2026-06-18
代码片段摘自各 PR 真实 diff , 已做教学性精简与中文注释。